Deep Learning to Optimize: Security-Constrained Unit Commitment With Uncertain Wind Power Generation and BESSs

Tong Wu, *Member, IEEE*, Ying-Jun Angela Zhang, *Fellow, IEEE*, and Shuoyao Wang, *Member, IEEE*

***Abstract*—This paper proposes a new model of scenario-based security-constrained unit commitment (SCUC) with BESSs. By formulating such a model as a mixed-integer programming (MIP) problem, we can obtain the optimal control strategy of units and BESSs to reduce the operating cost. To solve this MIP with the proposed model, we propose a new learning-based approach to tackle the SCUC problem. The proposed convolutional neural network (CNN)-based SCUC algorithm (CNN-SCUC) has two main stages. First, CNN-SCUC trains a CNN to obtain solutions to the binary variables corresponding to unit commitment decisions. Then, the continuous variables corresponding to unit power outputs are solved by a small-scale convex optimization problem. In contrast to existing work, CNN-SCUC eliminates the need of explicitly considering the scenario-based security constraints in the optimization problem, and thus greatly reduces the computational complexity. The average gap to the optimal solution is as small as 0.0267%. The algorithm is also scalable in the sense that the computational time is reduced from about 1236.32 seconds to 0.8379 seconds in a 10-unit and 200-scenario system. Besides, the computation time remains almost constant when the number of scenarios increases. Case studies show that compared with the traditional scenario-based SCUC model, more than 4.70% operating cost reduction is achieved by incorporating BESSs in the system.**

***Index Terms*—Deep learning, security-constrained unit commitment.**

NOMENCLATURE

Sets

$\mathcal{J}$	Set of indices of the generating units (number of units J).
$\mathcal{S}$	Set of scenarios (number of scenarios S).
$\mathcal{K}$	Set of indices of the time periods (Thermal units).
$\mathcal{K}_w$	Set of indices of the time periods (wind powers).
$\mathcal{K}_b$	Set of indices of the time periods (BESSs).

Abbreviation

CNN	Convolutional Neural Network.
SCUC	Security-constrained unit commitment.
CNN-SCUC	The CNN-based SCUC algorithm.
UC	Unit commitment.

Constants

$\bar{a}_j, \bar{b}_j, V_1, \bar{c}_j$	Coefficients of the quadratic production cost function of unit j
CC_j	Shutdown cost of unit j.
$D(k)$	Load demand in period k.
DT_j	Minimum down time of unit j.
G_j	Number of periods unit j must be initially online due to its minimum up time constraints.
K_j^t	Cost of the interval t of the stair-wise startup cost function of unit j.
L_j	Number of periods unit j must be initially offline due to its minimum down time contraint.
ND_j	Number of intervals of the s startup cost function of unit j.
NL_j	Number of segments of the piecewise linear production cost function of unit j.
$\overline{P}_j$	Capacity of unit j.
$\underline{P}_j$	Minimum power output of unit j.
$R(k)$	Spinning reserve requirement in period k.
RD_j	Ramp-down limit of unit j.
RU_j	Ramp-up limit of unit j.
$S_j(0)$	Number of periods unit j has been offline prior to the first period of the time span (end of period 0).
$Soc(ini)$	The initial value of State of Charge.
SD_j	Shutdown ramp limit of unit j.
SU_j	Startup ramp limit of unit j.
T	Number of periods of the time span of thermal units.
T_w	Number of periods of the time span of wind powers.
T_b	Number of periods of the time span of BESSs.
U_j^0	Number of periods unit j has been online prior to the first period of the time span (end of period 0).
UT_j	Minimum up time of unit j.
$V_j(0)$	Initial commitment state of unit j (1 if it is online, 0 otherwise).
$\overline{P}_b^{ch}$	The upper bound of BESSs charging powers.
$\underline{P}_b^{ch}$	The lower bound of BESSs charging powers.

Manuscript received December 14, 2020; revised May 9, 2021 and July 23, 2021; accepted August 21, 2021. Date of publication August 27, 2021; date of current version December 16, 2021. This work was supported in part by the Research Grants Council of Hong Kong through General Research Funding Project under Grant 14208017, and in part by the National Natural Science Foundation of China Project under Grant 62101336. Paper no. TSTE-01302-2020. *(Corresponding author: Shuoyao Wang).*

Tong Wu and Ying-Jun Angela Zhang are with the Department of Information Engineering, The Chinese University of Hong Kong, New Territories, Hong Kong (e-mail: tw385@cornell.edu; yjzhang@ie.cuhk.edu.hk).

Shuoyao Wang is with the College of Electronic and Information Engineering, Shenzhen University, Shenzhen 518060, China (e-mail: sywang@szu.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TSTE.2021.3107848.

Digital Object Identifier 10.1109/TSTE.2021.3107848

$\overline{P}_b^{dis}$	The upper bound of BESSs discharging powers.
$\underline{P}_b^{dis}$	The lower bound of BESSs discharging powers.
$\overline{Soc}$	The upper bound of Soc.
$\underline{Soc}$	The lower bound of Soc.
η_{ch}	Battery charge efficiency.
η_{dis}	Battery discharge efficiency.
$W^0(k)$	Wind power at time k.
$W^s(k)$	Wind power at time k in scenario s.
α	Reserve factor of wind powers.
Variables	
$C_j^d(k)$	Shutdown cost of unit j at time k.
$C_j^p(k)$	Production cost of unit j at time k.
$C_j^u(k)$	Startup cost of unit j at time k.
$p_j(k)$	Power output of unit j at time k.
$\overline{p}_j(k)$	Maximum feasible generation of unit j at time k.
$p_j^s(k)$	Power output of unit j at time k in scenario s.
$v_j(k)$	Binary variable that is equal to 1 if unit j is online at time k and 0 otherwise.
$p_b(k)$	Power output of BESSs at time k.
$p_b^{ch}(k)$	Power output of BESSs at time k when charging.
$p_b^{dis}(k)$	Power output of BESSs at time k when discharging.
$p_b^{s,ch}(k)$	Power output of BESSs at time k when charging in scenario s.
$p_b^{s,dis}(k)$	Power output of BESSs at time k when discharging in scenario s.
$b(k)$	Binary variable that is equal to 1 if BESSs are discharging at time k and 0 if BESSs are charging at time k.
$b^s(k)$	Binary variable in scenario s that is equal to 1 if BESSs are discharging at time k and 0 otherwise.
$Soc(k)$	State of charge of BESSs at time k.
$Soc^s(k)$	State of charge of BESSs at time k in scenario s.
$\mathbf{V}$	Unit commitment decisions, where the (jth, kth) entry of matrix $\mathbf{V}$ is $v_j(k)$.
$\mathbf{P}$	Matrix of power generation, where the (jth, kth) entry of matrix $\mathbf{P}$ is $p_j(k)$.
$\mathbf{P}^s$	Matrix of power generation in scenario s, where the (jth, kth) entry of matrix $\mathbf{P}^s$ is $p_j^s(k)$.

I. Introduction

The high penetration of wind power brings significant challenges to the economic and reliable operation of electric power systems. The main challenge is that the intermittent nature of wind power requires extra operation flexibility to address the inherent variability and uncertainty [1]–[3]. The flexibility issue can be addressed from both control and operation perspectives. From the control perspective, fast-response battery energy storage systems (BESSs) have recently been advocated as excellent candidates for facilitating the utilization of intermittent renewable sources [4]. From the operation perspective, the security-constrained unit commitment (SCUC) model has been introduced to strategically schedule the flexible resources to accommodate the uncertainty and variability of the wind power generation [5], [6]. The SCUC model is widely adopted to describe uncertainties using scenario approximation approaches [7].

In the current literature, scenario approximation is widely adopted to model the security constraints in SCUC models [6], [8], [9]. The key idea of scenario approximation is to generate a set of samples of random wind power generation and approximate the operational constraints with a set of constraints corresponding to the sampled scenarios [6]. The main challenges of scenario-based methods include: 1) high computational complexity. A large number of sampled scenarios are needed to achieve a reasonable approximation accuracy. This results in a large number of constraints in the optimization problem; 2) NP-hardness. The unit commitment variables are binary integer variables. This leads to mixed-integer programming (MIP) problems, which are in general NP-hard [10].

To overcome the first challenge, existing work has mainly focused on scenario reduction techniques (SRT). SRT reduces the number of constraints by selecting a limited number of typical scenarios [6], [11]–[15]. For example, an improved scenario mapping technology was proposed in [11] to model the cross-correlation among different trajectories. This method has a better balance between preserving uncertain information and model complexity. By measuring the distance between scenarios based on probability metrics, Ref. [6] eliminates scenarios with very low probability and aggregates scenarios that are close in distance. In [12], scenarios are grouped into partitions using a k-means clustering technique. Then, each partition of scenarios is reduced to the center of the partition. Noticeably, scenario reduction inevitably causes loss of information about the uncertain events. Ref. [15] proposed an optimal scenario selection method. However, this method is only applicable in linear programming problems. To address the second challenge, various techniques have been adopted to accelerate the solution to the large-scale SCUC problems. Among these methods, decomposition and distributed algorithms are widely adopted [1], [12], [16]–[18]. However, these methods heavily rely on the decomposability structure [19], [20]. For example, the Benders decomposition method in [6] requires the problem to be linear. However, scenario-based SCUC problems are in fact nonlinear [20].

Recently, machine learning methods have been recognized as a prominent technique to solve optimal power flow (OPF) problems [21]–[23]. For example, Ref. [21] solves the OPF problem by utilizing neural networks to predict the continuous optimal solutions. Likewise, security-constrained DC optimal power flow problems are considered in [22], where neural networks are employed to infer the optimal power generation. Notably, unit commitment decisions are not taken into account in [21], [22]. Ref. [23] proposed a learning-based method to address the security-constrained unit commitment problems. To simplify the problem, [24] omits many constraints that exist in practical generation stations, including generation limits and ramping constraints, minimum up and down time constraints, and the BESS integration.

In this paper, we formulate a scenario-based SCUC problem with fast-response BESSs to facilitate the incorporation of intermittent and stochastic wind powers. We develop a convolutional neural network (CNN)-based SCUC (CNN-SCUC) algorithm to solve the large-scale scenario-based SCUC with BESSs. The proposed CNN-SCUC has two inherent advantages over existing methods. On the one hand, it greatly reduces the computational complexity. This is because the CNN, once trained, can solve the binary variables corresponding to unit commitment decisions very quickly. Moreover, the CNN takes the sampled scenarios as input, and therefore the output binary solutions are likely to be feasible to the scenario-based security constraints. As a result, a large number of security constraints can be eliminated from the original optimization problem, and the continuous variables corresponding to unit power outputs can be obtained by solving a small-scale optimization problem. On the other hand, CNN-SCUC is data-driven and model-free, and does not require any decomposability of the SCUC model. Extensive numerical results show that CNN-SCUC obtains close-to-optimal solutions. The performance gap between the solution obtained by CNN-SCUC and the optimal one is as small as 0.0267% on average.

The remainder of this paper is organized as follows: Section II introduces the system model and problem formulation. Section III presents the proposed CNN-SCUC algorithm. Extensive case studies are presented in Section IV. Finally, Section V concludes the paper.

II. System Model and Problem Formulation

In this section, we present a scenario-based SCUC formulation that schedules units on an hourly basis. Suppose that the on-off decision over a K-hour period is described by a length K vector $\mathbf{v}_j = [v_j(1), \ldots, v_j(K)]$. A certain unit is either ON ($v_j(k) = 1$) or OFF ($v_j(k) = 0$) at each hour.

A. Security-Constrained Unit Commitment With Uncertainty Wind Power Model

The scenario-based SCUC problem with uncertain wind power is formulated as

$$\min_{\mathbf{P},\mathbf{P}^s,\mathbf{V}} \quad \sum_{k\in\mathcal{K}}\sum_{j\in\mathcal{J}} C_j^p(k) + C_j^u(k) + C_j^d(k) \tag{1a}$$

$$\text{s.t.} \quad (3)\text{–}(7), (11)\text{–}(36), \tag{1b}$$

where $C_j^p(k)$, $C_j^u(k)$ and $C_j^d(k)$ denote the fuel costs, startup and shutdown costs of individual units at hour k, respectively. Note that the power generation matrices $\mathbf{P}$ and $\mathbf{P}^s$ consist of continuous variables and the unit commitment matrix $\mathbf{V}$ consists of binary variables.

B. Objective Function

The three terms in the objective function are explained in the following.

1) Fuel Cost: The fuel cost of unit j is a quadratic function of power generation $p_j(k)$ and $v_j(k)$.

$$C_j^p(k) = \bar{a}_j v_j(k) + \bar{b}_j p_j(k) + \bar{c}_j p_j^2(k), \forall j \in \mathcal{J}, \forall k \in \mathcal{K}. \tag{2}$$

2) Startup Cost: Since the time span has been discretized into hourly periods (e.g., the example in Fig. 1), the startup cost is also a discrete-time function [24]. The startup cost is expressed as

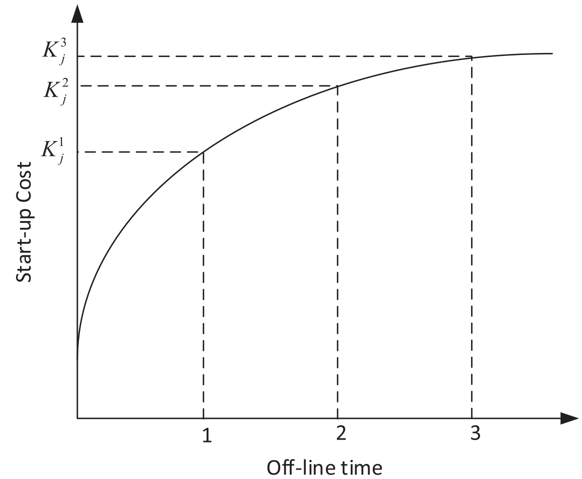

Fig. 1. Exponential, discrete, and stairwise startup cost functions.

$$\begin{aligned} C_j^u(k) = \max_t\{[K_j^t(v_j(k) - \sum_{n=1}^{t} v_j(k-n))]^+\},\\ \forall j \in \mathcal{J}, \forall k \in \mathcal{K}, \forall t = 1\ldots ND_j. \end{aligned} \tag{3}$$

3) Shutdown Cost: A constant shutdown cost CC_j is introduced in [24], and is shown as follows.

$$C_j^d(k) = [CC_j(v_j(k-1) - v_j(k))]^+, \forall j \in \mathcal{J}, \forall k \in \mathcal{K}. \tag{4}$$

C. Thermal Constraints

1) Generation Constraints: The generation constraints listed below include the system power balance constraints in (5) and system reserve requirements in (6).

$$\begin{aligned} \sum_{j\in J} p_j(k) + p_b(k_b) + W^0(k_w) = D(k),\\ \forall k \in \mathcal{K}, \forall k_w \in \mathcal{K}_w, \forall k_b \in \mathcal{K}_b. \end{aligned} \tag{5}$$

$$\begin{aligned} \sum_{j\in J}[\overline{P}_j v_j(k) - p_j(k)] \geq R(k)\\ +\alpha W^0(k_w), \forall k \in \mathcal{K}, \forall k_w \in \mathcal{K}_w. \end{aligned} \tag{6}$$

where $R(k)$ and $\alpha W^0(k_w)$ denote the spinning reserve in period k and additional wind reserve in period k_w, respectively.

2) Generation Limits and Ramping Constraints: The generation limits of each unit for each period are set as:

$$\underline{P}_j v_j(k) \leq p_j(k) \leq \overline{p}_j(k), \forall k \in \mathcal{K}, \forall j \in \mathcal{J}, \tag{7a}$$

$$0 \leq \overline{p}_j(k) \leq \overline{P}_j v_j(k), \forall k \in \mathcal{K}, \forall j \in \mathcal{J}, \tag{7b}$$

where $\overline{p}_j(k)$ denotes maximum feasible generation of thermal unit j at period k. Note that $C_j^p(k) = 0$ if $v_j(k) = 0$. The power generation $p_j(k)$ is also constrained by the ramp-up and startup ramp rates (8), shutdown ramp rates (9), and ramp-down limits

(10).

$$\overline{p}_j(k) \leq p_j(k-1)+ \max\{RU_j v_j(k-1) + SU_j[v_j(k) - v_j(k-1)], 0\} +\overline{P}_j[1 - v_j(k)], \forall k \in \mathcal{K}, \forall j \in \mathcal{J}. \tag{8}$$

$$\overline{p}_j(k) \leq \overline{P}_j v_j(k+1) + \max\{SD_j[v_j(k) - v_j(k+1)], 0\}, \forall k = 1 \ldots T-1, \forall j \in \mathcal{J}. \tag{9}$$

$$p_j(k-1) - p_j(k) \leq \max\{RD_j v_j(k) + SD_j[v_j(k-1) - v_j(k)], 0\} +\overline{P}_j[1 - v_j(k-1)], \forall k \in \mathcal{K}, \forall j \in \mathcal{J}. \tag{10}$$

In particular, the ramp-up and startup ramp rates (8) have different physical meanings:

- When unit j switches from the off status to the on status (i.e., $v_j(k-1) = 0$ and $v_j(k) = 1$), we have $\overline{p}_j(k) \leq SU_j$, where SU_j denotes the startup ramp limit of unit j.
- When unit j takes multiple hours to start (e.g., $v_j(k-1) = 1$ and $v_j(k) = 1$), we have $\overline{p}_j(k) \leq p_j(k-1) + RU_j$, where RU_j denotes the ramp-up limit of unit j.

In most practical scenarios, the ramp-down limit of unit j (RD_j) is equal to the shutdown ramp limit of unit j (SD_j); the ramp-up limit of unit j (RU_j) is equal to the startup ramp limit of unit j (SU_j); and the maximum power output of unit j ($\overline{P}_j$) is always larger than or equal to SD_j [14]. Therefore, (8) – (10) can be simplified to the following mixed-integer linear constraints:

$$\overline{p}_j(k) \leq p_j(k-1) + RU_j v_j(k-1)+ SU_j[v_j(k) - v_j(k-1)] + \overline{P}_j[1 - v_j(k)], \forall k \in \mathcal{K}, \forall j \in \mathcal{J}. \tag{11}$$

$$\overline{p}_j(k) \leq \overline{P}_j v_j(k+1) + SD_j[v_j(k) - v_j(k+1)], \forall k = 1 \ldots K-1, \forall j \in \mathcal{J}. \tag{12}$$

$$p_j(k-1) - p_j(k) \leq RD_j v_j(k)+ SD_j[v_j(k-1) - v_j(k)] + \overline{P}_j[1 - v_j(k-1)], \forall k \in \mathcal{K}, \forall j \in \mathcal{J}. \tag{13}$$

3) Minimum up and Down Time Constraints: The minimum up time constraints are as follows:

$$\sum_{k=1}^{G_j}[1 - v_j(k)] = 0, \forall j \in \mathcal{J}, \tag{14}$$

$$\sum_{n=k}^{k+UT_j-1} v_j(n) \geq UT_j[v_j(k) - v_j(k-1)], \forall j \in \mathcal{J}, \forall k = G_j + 1 \ldots K - UT_j + 1, \tag{15}$$

$$\sum_{n=k}^{T}[v_j(n) - [v_j(k) - v_j(k-1)]] \geq 0, \forall j \in \mathcal{J}, \forall k = K - UT_j + 2 \ldots T, \tag{16}$$

where $G_j = \min\{T, [UT_j - U_j^0]V_j(0)\}$. Note that G_j is the number of initial periods during which unit j must be online. Specifically, (14)–(16) represent the initial state, the intermediate time periods of the planning period, and the final state, respectively. Likewise, the minimum down time constraints are

$$\sum_{k=1}^{L_j}[v_j(k)] = 0, \forall j \in \mathcal{J}, \tag{17}$$

$$\sum_{n=k}^{k+DT_j-1} [1 - v_j(n)] \geq DT_j[v_j(k-1) - v_j(k)], \forall j \in \mathcal{J}, \forall k = L_j + 1 \ldots K - DT_j + 1, \tag{18}$$

$$\sum_{n=k}^{T}[1 - v_j(n) - [v_j(k-1) - v_j(k)]] \geq 0, \forall j \in \mathcal{J}, \forall k = K - DT_j + 2 \ldots K, \tag{19}$$

where $L_j = \min\{T, [DT_j - S_j(0)][1 - V_j(0)]\}$. Note that L_j is the number of initial periods during which unit j must be offline.

D. BESS Constraints

The constraints of the BESSs are as follows.

$$p_b(k_b) = p_b^{dis}(k_b) - p_b^{ch}(k_b), \forall k_b \in \mathcal{K}_b. \tag{20}$$

$$\underline{P}_b^{dis} b(k_b) \leq p_b^{dis}(k_b) \leq \overline{P}_b^{dis} b(k_b), \forall k_b \in \mathcal{K}_b. \tag{21}$$

$$\underline{P}_b^{ch}(1 - b(k_b)) \leq p_b^{ch}(k_b) \leq \overline{P}_b^{ch}(1 - b(k_b)), \forall k_b \in \mathcal{K}_b. \tag{22}$$

$$SoC(k_b) = SoC(k_b - 1) + \eta \Delta t p_b^{ch}(k_b) - 1/\eta \Delta t p_b^{dis}(k_b), \forall k_b = 2 \ldots T_b. \tag{23}$$

$$\underline{SoC} \leq SoC(k_b) \leq \overline{SoC}, \forall k_b \in \mathcal{K}_b. \tag{24}$$

$$SoC^s(k_b) = SoC^s(1) = SoC(ini), k_b = T_b. \tag{25}$$

$$b(k_b) \in \{0, 1\}, \forall k_b \in \mathcal{K}_b. \tag{26}$$

Here, the BESS extracts power from the power grid if $b(k_b) = 0$, and injects power into the grid if $b(k_b) = 1$. The power injection and power extraction are restricted by constraints (21) and (22), respectively. Constraint (23) indicates that the state of charge (SoC) of BESSs depends on the previous SoC, the injected and extracted power for the time interval multiplied by their respective efficiencies. Constraint (24) bounds the SoC of BESSs. Constraint (25) means that the SoC of BESSs must recover to the initial value for the next scheduling horizon.

E. Security Constraints

We consider the volatility of wind power generation in SCUC by simulating a set of wind power generation scenarios, and ensure that the unit commitment solution is feasible for all

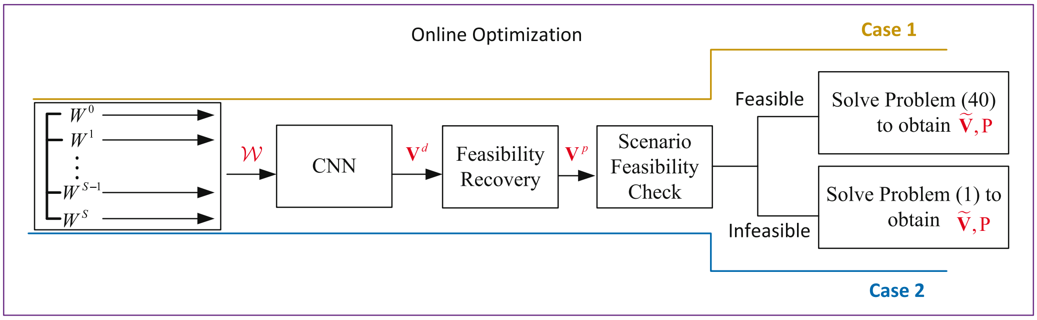

Fig. 2. The flowchart of CNN-SCUC.

scenarios. The resulting security constraints are given by

$$\sum_{j\in J} p_j^s(k) + p_b^s(k_b) + W^s(k_w) = D(k), \quad \forall k \in \mathcal{K}, \forall k_w \in \mathcal{K}_w, \forall k_b \in \mathcal{K}_b, \forall s \in \mathcal{S}. \tag{27}$$

$$\sum_{j\in J} [\overline{P}_j v_j(k) - p_j^s(k)] \geq R(k) + \alpha W^s(k_w), \quad \forall k \in \mathcal{K}, \forall k_w \in \mathcal{K}_w, \forall s \in \mathcal{S}. \tag{28}$$

$$\underline{P}_j v_j(k) \leq p_j^s(k) \leq \overline{P}_j v_j(k), \forall k \in \mathcal{K}, \forall j \in \mathcal{J}, \forall s \in \mathcal{S}. \tag{29}$$

$$p_b^s(k_b) = p_b^{s,dis}(k_b) - p_b^{s,ch}(k_b), \forall k_b \in \mathcal{K}_b, \forall s \in \mathcal{S}. \tag{30}$$

$$\underline{P}_b^{s,dis} b^s(k_b) \leq p_b^{s,dis}(k_b) \leq \overline{P}_b^{s,dis} b^s(k_b), \quad \forall k_b \in \mathcal{K}_b, \forall s \in \mathcal{S}. \tag{31}$$

$$\underline{P}_b^{s,ch}(1 - b^s(k_b)) \leq p_b^{s,ch}(k_b) \leq \overline{P}_b^{s,ch}(1 - b^s(k_b)), \quad \forall k_b \in \mathcal{K}_b, \forall s \in \mathcal{S}. \tag{32}$$

$$SoC^s(k_b) = SoC^s(k_b - 1) + \eta_{ch}\Delta t p_b^{s,ch}(k_b) - 1/\eta_{dis}\Delta t p_b^{s,dis}(k_b), \forall k_b = 2 \ldots T_b, \forall s \in \mathcal{S}. \tag{33}$$

$$\underline{SoC} \leq SoC^s(k_b) \leq \overline{SoC}, \forall k_b \in \mathcal{K}_b, \forall s \in \mathcal{S}. \tag{34}$$

$$SoC^s(k_b) = SoC^s(1) = SoC(ini), k_b = T_b, \forall s \in \mathcal{S}. \tag{35}$$

$$b^s(k_b) \in \{0, 1\}, \forall k_b \in \mathcal{K}_b, \forall s \in \mathcal{S}. \tag{36}$$

The larger the scenario set $\mathcal{S}$, the more robust the unit commitment solution is to wind power volatilities. Needless to say, there exists a tradeoff between the robustness and the efficiency (i.e., the objective function value) of the unit commitment solution. Noticeably, constraints (27) — (36) ensure that in each scenario, the binary variables $\mathbf{V} = [\mathbf{v}_1, \ldots, \mathbf{v}_J]^T$ for units are capable of accommodating the random wind power generation. The set of continuous variables corresponding to unit power outputs $\mathbf{P} = [\mathbf{p}_1, \ldots, \mathbf{p}_J]^T$, where $\mathbf{p}_j = [p_j(1), \ldots, p_j(K)]$ and the set of continuous variables corresponding to unit power outputs $\mathbf{P}^s = [\mathbf{p}_1^s, \ldots, \mathbf{p}_J^s]^T$ in scenario s, where $\mathbf{p}_j^s = [p_j^s(1), \ldots, p_j^s(K)]$ and $s \in \mathcal{S}$. Under different scenarios, $\mathbf{P}^s$ are different but $\mathbf{V}$ is the same for all scenarios.

The above optimization Problem (1) is a large-scale MIP problem. The complexity of the problem increases. In addition, the presence of binary variables makes this problem hard to solve.

III. CNN-Based Approximation to SCUC Algorithm

In this section, we propose CNN-SCUC to solve Problem (1) efficiently. As illustrated in Fig. 2, CNN-SCUC first utilizes the CNN to generate UC decisions by taking wind power samples as input. Then, it solves a small-scale convex optimization problem in (40) to obtain the solutions to the continuous variables. In between, the feasibility check is to deal with a small percentage of infeasible outputs from the CNN.

A. Notations

In Fig. 2, we denote by $\mathbf{V}^d \in \mathbb{R}^{J\times K}$ the output UC decisions that are predicted by CNN. In particular, $\mathbf{V}^d = [\mathbf{v}_1^d, \ldots, \mathbf{v}_J^d]^T$, where $\mathbf{v}_j^d = [v_j^d(1), \ldots, v_j^d(K)]$. Similarly, we denote by $\mathbf{V}^p$ the projected unit commitment decisions, which are obtained by mapping $\mathbf{V}^d \in \mathbb{R}^{J\times K}$ into the feasible set of (14)–(19). At last, $\widetilde{\mathbf{V}} \in \mathbb{R}^{J\times K}$ are the final unit commitment decisions. In Fig. 3, $\mathbf{V}^* \in \mathbb{R}^{J\times K}$ are the optimal unit commitment decisions by solving Problem 1 using the branch-and-cut algorithm. In particular, $\widetilde{\mathbf{V}}$ are equal to $\mathbf{V}^p$ if $\mathbf{V}^p$ are feasible to constraints (27)–(29), and $\mathbf{V}^*$ otherwise. The jth row of matrices $\mathbf{V}^d$, $\mathbf{V}^p$, $\widetilde{\mathbf{V}}$, and $\mathbf{V}^*$ are denoted by $\mathbf{v}_j^d$, $\mathbf{v}_j^p$, $\widetilde{\mathbf{v}}_j$, and $\mathbf{v}_j^*$, respectively. The (jth, kth) entry of matrices $\mathbf{V}^d$, $\mathbf{V}^p$, $\widetilde{\mathbf{V}}$, and $\mathbf{V}^*$ are denoted by $v_j^d(k)$, $v_j^p(k)$, $\widetilde{v}_j(k)$, and $v_j^*(k)$, respectively.

B. Training of the CNN

Before CNN-SCUC is implemented, the CNN is trained by feeding the pairs of wind power samples and optimal UC decisions, as shown in Fig. 3. Recall that the input of the network is the predicted wind power generation and the set of simulated wind power generation scenarios $\mathcal{S}$. The output $v_j(k)$ is a binary variable that is equal to 1 if unit j is online at time k and 0 otherwise.

As a common practice, we use three hidden layers to reduce the generalization error. In addition, we use ReLU as the activation function in the hidden layers, where ReLU is mathematically expressed as $y = \max\{x, 0\}$. In the output layer, we use a sigmoid activation function, where the output y and input x of a neuron are related by $y = 1/(1 + e^{-x})$. In particular, for each $v_j(k), \forall k \in \mathcal{K}, \forall s \in \mathcal{S}$, we set $v_j(k) = 1$ if $y_j(k) \geq 0.5$, and 0 otherwise.

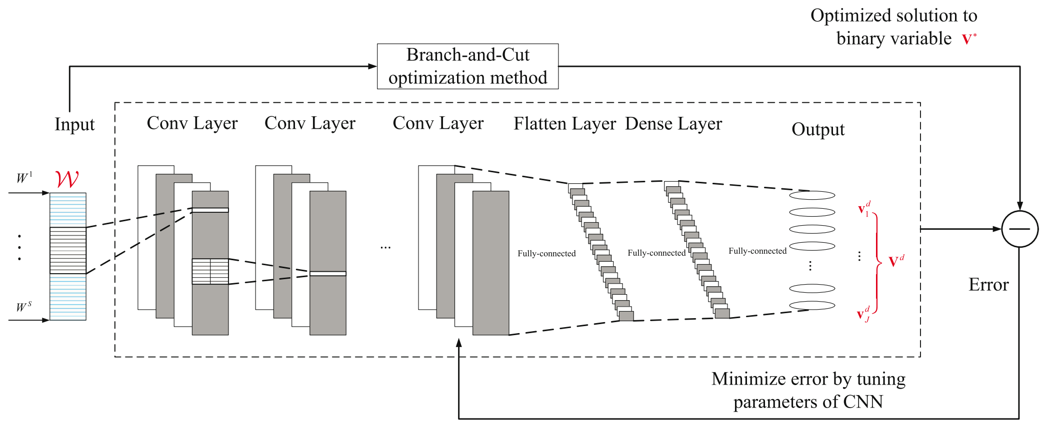

Fig. 3. Training Stage.

To train the CNN, we first generate a set of M labeled data $\{\mathcal{W}_l, \mathbf{V}_l^*\}$ for $l = 1, \ldots, M$, where $\mathcal{W}_l$ is the wind power sample and $\mathbf{V}_l^*$ denotes the optimal UC decisions. In particular, the generation of wind power samples is illustrated as follows.

- The predicated wind powers $\boldsymbol{W}^0$ are obtained from the power generation data from NREL Wind [25].[1]
- The simulated wind power scenarios $\boldsymbol{W}^i$ $(i = 1, \ldots, S)$ are generated according to a normal distribution with the expected value equal to $\boldsymbol{W}^0$ and the standard deviation (volatility) of 10% of $\boldsymbol{W}^0$. In particular, the Monte Carlo simulation generates a large number of scenarios subject to the above normal distribution. Then each scenario $\boldsymbol{W}^i$ $(i = 1, \ldots, S)$ is sampled by the Latin hypercube sampling (LHS) method [25].

In this paper, we use the LHS from [25] to reduce the number of runs for a simple Monte Carlo, achieving a reasonably accurate Gaussian distribution. One wind power sample $\mathcal{W}$ contains the predicted wind powers $\boldsymbol{W}^0$ and simulated scenarios $\boldsymbol{W}^i$ $(i = 1, \ldots, S)$, which is denoted by $\mathcal{W} = [\boldsymbol{W}^0, \boldsymbol{W}^1, \ldots, \boldsymbol{W}^S]$. Specifically, $\boldsymbol{W}^i$ $(i = 0, \ldots, S)$ is described by a length K vector over a K-hour period in one day, $i.e.$, $\boldsymbol{W}^i = [W^i(1), \ldots, W^i(K)]$. For each wind power sample $\mathcal{W}$, we calculate the optimal UC decisions $\mathbf{V}^*$ by the branch-and-cut algorithm [26]. We treat $\{\mathcal{W}, \mathbf{V}^*\}$ as a pair of labeled data. In total, we generate M set pairs, denoted by $\{\mathcal{W}_l, \mathbf{V}_l^*\}$, as the labeled dataset to train the CNN.

Overall, the input-output relation of the CNN is expressed as

$$f_\theta : \mathcal{W}_l \mapsto \mathbf{V}_l^*, \forall l = 1, \ldots, M \tag{37}$$

where $\mathbf{V}_l^* \in \{0, 1\}^{J \times K}$. The training process is introduced as follows. First of all, we use the mini-batch gradient descent to train the proposed CNN using the pairs $\{\mathcal{W}_l, \mathbf{V}_l^*\}_{l=1,\ldots,M}$. In the t-th iteration, we randomly samples a mini-batch from the training data with the batch size of $|\mathcal{T}_t|$, i.e., $\{\mathcal{W}_\tau, \mathbf{V}_\tau^* | \tau \in \mathcal{T}_t\}$. The parameters θ of the CNN, including the filters $\boldsymbol{h}$, weights $\boldsymbol{w}$, and biases $\boldsymbol{b}$ in each layer, are updated through the Adam optimizer [27] by minimizing the cross-entropy loss between the predicted output of the CNN and the labeled output $\mathbf{V}_l^*$.

[1] [Online]. Available: https://www.nrel.gov/grid/wind-integration-data.html

Algorithm 1: CNN-SCUC.

1 **for** $l = 1 : \infty$ **do**
2   Feed $\mathcal{W}_l$ into the trained CNN to obtain the unit commitment decisions $\mathbf{V}_l^d$;
3   **if** $\mathbf{V}_l^d$ *is infeasible to constraints* (14) - (19) **then**
4   Project $\mathbf{V}_l^d$ to the minimum up and down time constraints (14) - (19) by solving (39) to obtain $\mathbf{V}_l^p$;
5   **else**
6         $\mathbf{V}_l^p = \mathbf{V}_l^d$;
7   **if** $\mathbf{V}_l^p$ *is feasible to constraints* (27) - (29) **then**
8   Solve Problem (40) to obtain $\mathbf{P}_l$ given $\mathbf{V}_l^p$;
9   **return** $\{\mathbf{P}_l, \widetilde{\mathbf{V}}_l = \mathbf{V}_l^p\}$;
10   **else**
11   Solve Problem (1) to obtain $\mathbf{P}_l$ and $\mathbf{V}_l^*$;
12   **return** $\{\mathbf{P}_l, \widetilde{\mathbf{V}}_l = \mathbf{V}_l^*\}$;

After multiple training epochs, we obtain the trained CNN f_{θ^*}. The detailed structure of the CNN can be found in our previous work [28].

C. Optimization - CNN-SCUC

Once the CNN is trained, we can use it to solve Problem (1) efficiently by the proposed CNN-SCUC. We provide the pseudo-code of the proposed CNN-SCUC in Algorithm 1. In particular, step 2 generates the UC decisions using the trained CNN. Feasibility recovery is conducted in steps 3 - 4 for the few solutions that are infeasible to constraints (14)–(19). The feasibility of scenario-based security constraints is checked in step 7. In most cases, $\mathbf{V}_l^p$ is feasible to the security constraints (27)–(36), and thus the constraints can be eliminated. This way, the original MIP problem is reduced to Problem (40). For rare cases when $\mathbf{V}_l^p$ is infeasible, we solve Problem (1) to obtain the optimal results in step 11. The details of the steps are elaborated in the following.

1) UC Decision Generation Steps (Step 2 of Algorithm 1): In this step, we first collect the predicted wind power data published by the NREL WIND Tookit and generate the simulated scenarios following the same distribution as in the training stage. Then, we feed $\mathcal{W}$ into the trained CNN and obtain the output unit commitment decisions $\mathbf{V}^d$.

2) The Feasibility Recovery and Scenario Feasibility Check Steps (Steps 3-6 of Algorithm 1): Note that both the predicted wind power generation and simulated power-generation scenarios are the input of the CNN. Therefore, the output UC decisions $\mathbf{V}^d$ are likely to satisfy both the minimum up and down constraints (14)–(19) and the security constraints (27)–(29). For the few infeasible solutions, we project them to the feasible set defined by (14)–(19) for simplicity, but ignore the security constraints (27)–(36). The projection is obtained by solving

$$\mathbf{V}^p = \arg\min_{\mathbf{V}_j} \left\|\mathbf{V}_j - \mathbf{V}_j^d\right\|_2^2 \tag{38a}$$

$$\text{s.t.} \quad (14) - (19), \tag{38b}$$

Since $\mathbf{V}^p$ is the Cartesian product of the subsets of each unit, $i.e.$, $\mathbf{V}^p = \mathbf{v}_1^p \times \cdots \times \mathbf{v}_J^p$, we can solve Problem (38) for each unit by parallelization. The subproblems of Problem (38) are introduced as follows. For each unit, we first check the minimum up and down time constraints (14)–(19). If these constraints are violated for a unit, we project the solution $\mathbf{v}_j^d$ to the feasible set (14)–(19) as follows.

$$\mathbf{v}_j^p = \arg\min_{\mathbf{v}_j} \left\|\mathbf{v}_j - \mathbf{v}_j^d\right\|_2^2 \tag{39a}$$

$$\text{s.t.} \quad (14) - (19), \tag{39b}$$

where $\mathbf{v}_j^p$ is the feasible solution that has the minimum Euclidean distance to $\mathbf{v}_j^d$. Usually solving Problem (39) for one unit is inexpensive. In addition, the projection process is seldom conducted because $\mathbf{V}^d$ satisfies the constraints (14)–(19) most of the time. The mapping process only involves (14)–(19) because the number of security constraints in (27)–(36) is much larger than the number of the minimum up and down constraints in (14)–(19). Besides, we find that the projected solutions satisfy the scenario-based security constraints in most cases, thanks to the fact that the scenarios are inputs of the CNN.

After obtaining $\mathbf{V}^p$, we need to check the feasibility of security constraints (27)–(36) for each scenario. Note that the constraints for different scenarios are decoupled given $\mathbf{V}^p$, and thus we have small-size feasibility check problems. In particular, if all the security constraints (27)–(36) for each scenario are satisfied by $\mathbf{V}^p$, we remove these constraints from the original MIP Problem (1).

3) The Desired Solution to SCUC: Given the unit commitment decisions $\mathbf{V}^p$, the original MIP Problem (1) is reduced to a convex optimization problem with continuous variables $\mathbf{P}$ only.

$$\min_{\mathbf{P}} \quad \sum_{k\in K}\sum_{j\in J} C_j^p(k) + C_j^u(k) + C_j^d(k) \tag{40a}$$

$$\text{s.t.} \quad (5)–(7), (11)–(13), \tag{40b}$$

$$v_j(k) = v_j^p(k), \forall k \in \mathcal{K}, \forall j \in \mathcal{J}. \tag{40c}$$

The problem can be efficiently solved using standard solvers, e.g., Gurobi. Note that Problem (40) has $J \times K$ continuous variables, and the original Problem (1) has $J \times K \times (S+1)$ continuous variables and $J \times K$ integer variables. Therefore, the size of Problem (40) is much smaller than the original problem, especially when S is large.

IV. Case Studies

In this section, we consider a case study with 10 units and 100 units over a 24-hour period. The data of the 10-unit and 100-unit systems are chosen in [14] and [24], respectively. We aim to demonstrate that: 1) the proposed model is capable of reducing the operating cost by incorporating BESSs into systems; 2) the proposed method drastically reduces the computational complexity compared with the traditional optimization methods; and 3) the optimal results obtained by the proposed CNN-SCUC are very close (or equal) to the optimal results obtained by off-the-shelf solvers.

A. Performance of the SCUC With BESS Model

We use the wind power generation data from NREL Wind [25]. The Latin hypercube sampling technique is employed to generate the wind power scenarios [6]. Besides, we set the upper bound of SoC as 0.9 and the lower bound of SoC as 0.3. Besides, the battery charge efficiency η_{ch} and discharge efficiency η_{dis} are both 0.95. The maximal wind power is set to be 480 MW because BESSs are capable of dealing with very large wind powers. We randomly choose ten days from NREL Wind to compute the operating cost. We use the same setting to get deterministic results for fair comparison. For comparison, we simulate the following six cases, where model 1 and model 2 correspond to Ref. [24] and Ref. [14], respectively, and models 4 – 6 correspond to our proposed SCUC with wind powers and BESSs.

- Model 1: Unit commitment with wind powers [24]. This model does not consider the scenario-based security constraints.
- Model 2: Security-constrained unit commitment with wind powers [14].
- Model 3: Unit commitment with wind powers and BESSs. The BESSs operate every hour ($k_b = k$). This model incorporates BESSs without the scenario-based security constraints.
- Model 4: Security-constrained unit commitment with wind powers and BESSs. The BESSs operate every hour ($k_b = k$).
- Model 5: Unit commitment with wind powers and BESSs. This model incorporates BESSs that operate every 5 minutes ($k_b = k_w$), but does not consider the scenario-based security constraints.
- Model 6: Security-constrained unit commitment with wind powers and BESSs. The BESSs operate every 5 minutes ($k_b = k_w$).

In TABLE I, the case studies show the advantages of security constraints and BESSs in the SCUC problem as follows: 1) Compared with model 1, model 2 considers the scenario-based security constraints. Therefore, although the operating cost of model 2 in each day is higher than that of model 1, the system of model 2 achieves more robust dispatch for accommodating wind power volatilities; 2) Compared with model 2, model 4 and model 6 achieve a lower operating by employing the BESSs. Specifically, model 4 and model 6 reduce the operating cost by

TABLE I
THE RESULTS OF COSTS

Cost	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7	Day 8	Day 9	Day 10
Model 1	499608.42	542025.45	420626.34	366567.79	543367.33	539932.97	566275.66	567670.37	496528.21	425206.12
Model 2	500884.75	542834.61	424042.55	388088.32	543487.71	540465.35	566275.66	567670.37	498287.37	430449.00
Model 3	493807.94	534687.42	413302.20	358608.26	535273.79	531809.62	557039.44	558583.42	489860.62	417673.33
Model 4	494039.29	534687.42	415008.56	360802.88	535273.79	532040.64	557039.44	558583.42	490646.88	418332.77
Model 5	494338.06	532355.24	404539.02	368920.60	535796.98	534165.32	557048.85	558583.42	486570.18	420075.58
Model 6	494731.30	532666.57	405017.28	370040.49	535796.98	534165.32	557048.85	558583.42	487215.06	420198.04

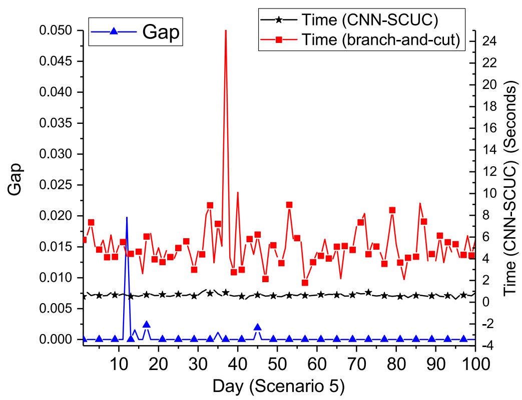

Fig. 4. The optimality gap and computation time (The number of scenarios is 5).

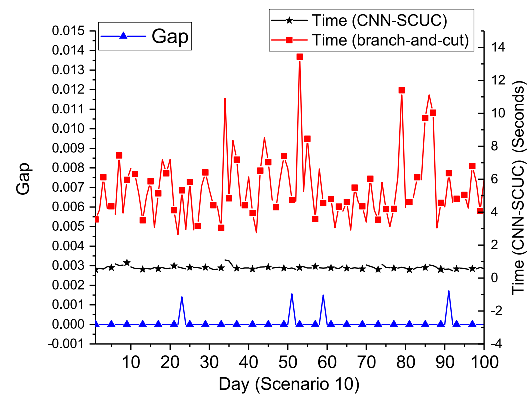

Fig. 5. The optimality gap and computation time (The number of scenarios is 10).

up to 2.18% and 4.70%, respectively, compared with model 2 on day 3. During one day, the BESSs are capable to store wind powers when wind powers are large and discharge them when wind powers are small. Therefore, the operating cost is reduced.

B. Optimality Gap

In this subsection, we test the proposed CNN-SCUC based on Model 4, where BESSs operate very hour. We generate 21377 samples of 5 scenarios and 10 scenarios, respectively. Recall that each sample represents the set of predicted and simulated wind power generations $\mathcal{W}_l$ ($l = 1, \ldots, M$). All the simulations are performed on a computer with a 3.4 GHz Quad-Core Intel Core i5 CPU and 24 GB memory. The branch-and-cut algorithm is implemented in the off-the-shelf solver, i.e. Gurobi, to obtain the optimal solutions for training and testing purposes. The training stage of the CNN is implemented in Python with Tensorflow 1.8.0. The training, validation, and testing data constitute 70%, 10% and 20% of 21377 set pairs $\{\mathcal{W}_l, \mathbf{V}_l^*\}$, respectively. The training batch size is 200, and the initial learning rate $L_0 = 1 * 10^{-3}$ with a patience of 5. Once trained, CNN remains valid as long as the system topology does not change. From a long-term perspective, the time cost of the training process can be omitted.

In Figs. 4 and 5, we investigate the performance of CNN-SCUC in terms of the gap from the optimal solution obtained by the branch-and-cut algorithm. We randomly choose 100 samples ($i.e.$, 100 days) from the testing datasets, including 5 scenarios and 10 scenarios, respectively. Let us denote z^* as the optimal value to the original problem. Likewise, let z_{UB} denote the objective function value obtained by the proposed CNN-SCUC. Here, the percentage optimality gap is calculated as gap $=100 \times (z_{UB} - z^*)/z_{UB}$ %. Figs. 4 and 5 show that the percentage optimality gap is very small. The average gaps are only on average 0.0267% and 0.00619% for scenario 5 and scenario 10, respectively. This indicates that the solution obtained by the proposed CNN-SCUC is very close to the global optimal solution.

C. CNN-SCUC in the 100-Unit System

We test the proposed algorithm in a large-scale unit commitment problem with 100 units [24]. We randomly choose 50 samples, $i.e.$, 50 days, from the testing datasets with 10 scenarios. In particular, the system includes 2400 binary variables to be predicted by the CNN. Likewise, the renewable energies are obtained from the power generation data from NREL Wind. The results in terms of optimality gap and computation time are shown in Fig. 6. The figure shows that proposed DSUCU significantly reduces the computation time. Besides, the maximal optimality gap is only 0.043%. This indicates that the proposed algorithm is scalable to a large-scale unit commitment power system.

D. Computational Complexity

The computational time of Algorithm 1 is different for two cases, shown in Fig. 2. From our numerical results, 92.15% of $\mathbf{V}_l^p$ in step 9 of Algorithm 1 are already feasible to the scenario-based security constraints (27)–(36), i.e., belong to Case 1 in Fig. 2. As such, the scenario-based security constraints can be simplified. Therefore, the computational complexity of Case 1 is very low. For another thing, the computational complexity of Case 2 is almost the same as the branch-and-cut algorithm.

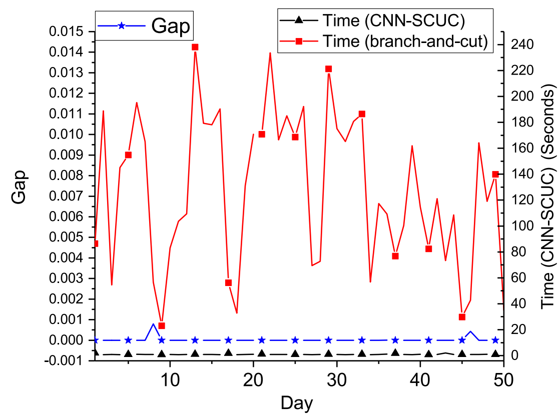

Fig. 6. The optimality gap and computation time in the 100-unit system (The number of scenarios is 10).

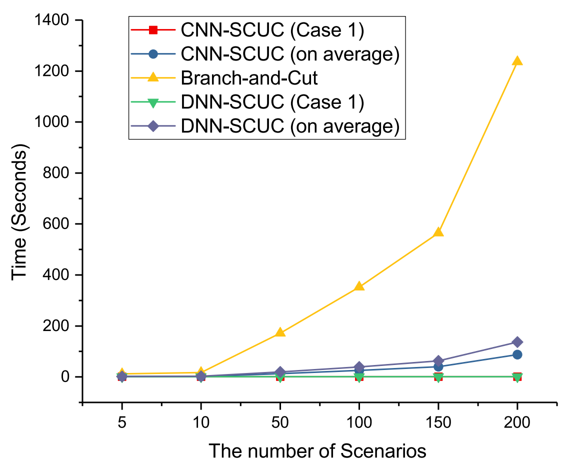

Fig. 7. The comparison of computation time.

In Fig. 4 - Fig. 5, with the same number of scenarios, the computation time by Gurobi varies greatly with different wind power generations. This is because Problem (1) is NP-hard, and the branch-and-cut algorithm in Gurobi that solves such a MIP problem with different parameters may have different branches and additional cutting planes. In the worst case, these algorithms cannot converge within a limited time. For example, Fig. 4 - Fig. 5 show that the maximal computation time by Gurobi is over 24 s and 13 s, respectively.

In Fig. 7, CNN-SCUC (Case 1) denotes the average time cost incurred in Case 1. CNN-SCUC (on average) denotes the average time cost incurred in both Case 1 and Case 2. CNN-SCUC (Case 1) shows that the CNN-SCUC algorithm solves Problem (1) within approximately constant computation time (i.e., 0.8379 seconds) regardless of the number of scenarios. In addition, Gurobi solves such a problem with increasing time as the number of scenarios increases. For example, with 200 scenarios, the computation time of Gurobi is 1236.32 seconds. Besides, another standard solver, e.g., Mosek, cannot converge within a limited time when it is applied into Problem (1) because of the NP-hard problem. Our proposed CNN-SCUC is a model-free algorithm, which means that it is practical and can applied into complex models.

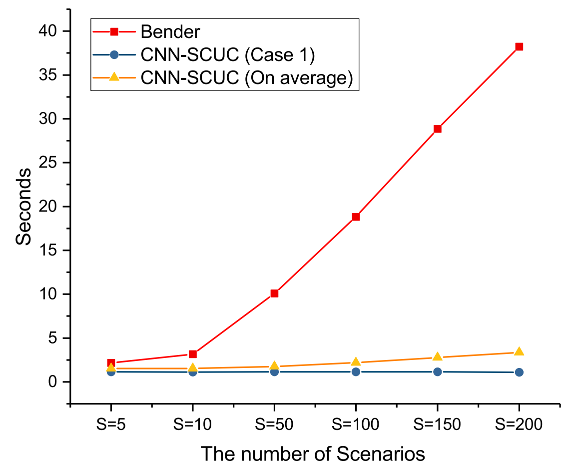

Fig. 8. The comparison of the proposed CNN-SCUC and the Benders decomposition method in terms of the computation time.

We also compare with the proposed CNN with the fully-connected DNN in the SCUC problem. In particular, we conduct the comparison by replacing the proposed CNN with a DNN. The benchmark algorithm with the DNN architecture is denoted as DNN-SCUC. In Fig. 7, we plot the comparison of the computation times versus the number of scenarios. For 200 scenarios, the CNN-SCUC method reduces the computation time from 136.74 seconds to 87.32 seconds in contrast to the DNN-SCUC method. This is because the DNN-SCUC method has lower accuracy rate compared with the CNN-SCUC method so that the DNN-SCUC method is more likely to undergo Case 2 in Fig. 2 of the revised manuscript. The proposed CNN structure achieves a higher accuracy by capturing the spatial-temporal dependency on unit commitment decisions over different units and time slots [28]. For example, when the wind powers are large, these small-scale units are more likely to be on the off status simultaneously due to the fuel cost minimization, e.g., Units 8-10 in the 10-unit system.

E. Comparison With the Benders Decomposition Method

For the fair comparison, we utilize both the proposed CNN-SCUC method and the Benders decomposition method for the SCUC problem without BESSs. In particular, the SCUC problem without BESSs is a special case of the proposed model (i.e., Problem (1)) by setting $\overline{P}_b^{dis} = \underline{P}_b^{dis} = 0$ and $\overline{P}_b^{ch} = \underline{P}_b^{ch} = 0$ in (21) and (22), and $\overline{P}_b^{s,dis} = \underline{P}_b^{s,dis} = 0$ and $\overline{P}_b^{s,ch} = \underline{P}_b^{s,ch} = 0$ in (31) and (32). In Fig. 8, the simulation results show that the proposed algorithm can reduce the computation time significantly, i.e., from 38.2276 seconds to 1.1021 seconds in our case study.

V. Conclusions

In this paper, a new model of scenario-based SCUC with BESSs was proposed and formulated as an MIP problem. We proposed a deep learning based method named CNN-SCUC to solve the large-scale SCUC problem. The proposed CNN-SCUC first trains a CNN to obtain the solutions to the binary variables corresponding to unit commitment decisions. With the solutions to the binary variables, the scenario-based security constraints are then eliminated because these constraints only bound the binary variables. At last, CNN-SCUC solves the continuous variables corresponding to unit power outputs by small-scale convex optimization. Case studies showed incorporating BESSs in the system reduces the operating cost by more than 4.70%. In addition, the proposed algorithm achieves close-to-optimal SCUC solutions with about 0.0267% gap from the optimal solutions. Besides, it significantly reduces the computation time from 1236.32 seconds to 0.8379 seconds in a 10-unit and 200-scenario system.

References

[1] A. Ahmadi, A. E. Nezhad, P. Siano, B. Hredzak, and S. Saha, "Information-gap decision theory for robust security-constrained unit commitment of joint renewable energy and gridable vehicles," *IEEE Trans. Ind. Inform.*, vol. 16, no. 5, pp. 3064–3075, May 2020.

[2] M. Shahidehpour, Y. Zhou, Z. Wei, G. Sun, and S. Chen, "Multistage robust look-ahead unit commitment with probabilistic forecasting in multi-carrier energy systems," *IEEE Trans. Sustain. Energy*, vol. 12, no. 1, pp. 70–82, Jan. 2021.

[3] T. Wu, C. Zhao, and Y. J. Zhang, "Distributed AC-DC optimal power dispatch of VSC-Based energy routers in smart microgrids," *IEEE Trans. Power Syst*., vol. 36, no. 5, pp. 4457–4470, Sep. 2021.

[4] H. Khorramdel, J. Aghaei, B. Khorramdel, and P. Siano, "Optimal battery sizing in microgrids using probabilistic unit commitment," *IEEE Trans. Ind. Inform.*, vol. 12, no. 2, pp. 834–843, Apr. 2016.

[5] F. Bavafa, T. Niknam, R. Azizipanah-Abarghooee, and V. Terzija, "A new biobjective probabilistic risk-based wind-thermal unit commitment using heuristic techniques," *IEEE Trans. Ind. Inform.*, vol. 13, no. 1, pp. 115–124, Feb. 2017.

[6] J. Wang, M. Shahidehpour, and Z. Li, "Security-constrained unit commitment with volatile wind power generation," *IEEE Trans. Power Syst.*, vol. 23, no. 3, pp. 1319–1327, Aug. 2008.

[7] Z. Jin, K. Pan, L. Fan, and T. Ding, "Data-driven look-ahead unit commitment considering forbidden zones and dynamic ramping rates," *IEEE Trans. Ind. Inform.*, vol. 15, no. 6, pp. 3267–3276, Jun. 2019.

[8] C. Wang *et al.*, "Robust risk-constrained unit commitment with large-scale wind generation: An adjustable uncertainty set approach," *IEEE Trans. Power Syst.*, vol. 32, no. 1, pp. 723–733, Jan. 2017.

[9] Y. Du, Y. Li, C. Duan, H. Gooi, and L. Jiang, "An adjustable uncertainty set constrained unit commitment with operation risk reduced through demand response," *IEEE Trans. Ind. Inform.*, vol. 17, no. 2, pp. 1154–1165, Feb. 2021.

[10] T. Wu, Y. J. Zhang, and X. Tang, "A VSC-Based BESS model for multi-objective OPF using mixed integer SOCP," *IEEE Trans. Power Syst.*, vol. 34, no. 4, pp. 2541–2552, Jul. 2019.

[11] E. Du, N. Zhang, C. Kang, and Q. Xia, "Scenario map-based stochastic unit commitment," *IEEE Trans. Power Syst.*, vol. 33, no. 5, pp. 4694–4705, Sep. 2018.

[12] I. Blanco and J. M. Morales, "An efficient robust solution to the two-stage stochastic unit commitment problem," *IEEE Trans. Power Syst.*, vol. 32, no. 6, pp. 4477–4488, Nov. 2017.

[13] Q. Zhai, X. Li, X. Lei, and X. Guan, "Transmission constrained UC with wind power: An all-scenario-feasible MILP formulation with strong nonanticipativity," *IEEE Trans. Power Syst.*, vol. 32, no. 3, pp. 1805–1817, May 2017.

[14] J. Wang *et al.*, "Wind power forecasting uncertainty and unit commitment," *Appl Energy*, vol. 88, no. 11, pp. 4014 – 4023, 2011.

[15] A. M. C. So and Y. J. A. Zhang, "Distributionally robust slow adaptive OFDMA with soft QoS via linear programming," *J. Sel. Areas Commun.*, vol. 31, no. 5, pp. 947–958, May 2013.

[16] Á. Lorca, X. A. Sun, E. Litvinov, and T. Zheng, "Multistage adaptive robust optimization for the unit commitment problem," *Oper. Res.*, vol. 64, no. 1, pp. 32–51, 2016. [Online]. Available: https://doi.org/10.1287/opre.2015.1456

[17] A. Kargarian, Y. Fu, and Z. Li, "Distributed security-constrained unit commitment for large-scale power systems," *IEEE Trans. Power Syst.*, vol. 30, no. 4, pp. 1925–1936, Jul. 2015.

[18] T. Ding *et al.*, "Duality-free decomposition based data-driven stochastic security-constrained unit commitment," *IEEE Trans. Sustain. Energy*, vol. 10, no. 1, pp. 82–93, Jan. 2019.

[19] D. P. Palomar and M. Chiang, "A tutorial on decomposition methods for network utility maximization," *J. Sel. Areas Commun.*, vol. 24, no. 8, pp. 1439–1451, Aug. 2006.

[20] R. Rahmaniani, T. G. Crainic, M. Gendreau, and W. Rei, "The benders decomposition algorithm: A literature review," *Eur. J. Oper. Res.*, vol. 259, no. 3, pp. 801 – 817, 2017.

[21] X. Pan, T. Zhao, M. Chen, and S. Zhang, "Deepopf: A deep neural network approach for security-constrained DC optimal power flow," *IEEE Trans. Power Syst*., vol. 36, no. 3, pp. 1725–1735, May 2021.

[22] A. Velloso and P. Van Hentenryck, "Combining deep learning and optimization for preventive security-constrained DC optimal power flow," *IEEE Trans. Power Syst*., vol. 36, no. 4, pp. 3618–3628, Jul. 2021.

[23] Á. S. Xavier, F. Qiu, and S. Ahmed, "Learning to solve large-scale security-constrained unit commitment problems," *INFORMS J. Comput.*, vol. 33, no. 2, pp. 739–756, 2021.

[24] M. Carrion and J. M. Arroyo, "A computationally efficient mixed-integer iinear formulation for the thermal unit commitment problem," *IEEE Trans. Power Syst.*, vol. 21, no. 3, pp. 1371–1378, Aug. 2006.

[25] C. Draxl, A. Clifton, B.-M. Hodge, and J. McCaa, "The wind integration national dataset (WIND) toolkit," *Appl Energy*, vol. 151, pp. 355–366, 2015.

[26] M. Conforti, G. Cornuejols, and G. Zambelli, *Integer Programming*. Berlin, Germany: Springer, vol. 271, 2014.

[27] D. P. Kingma and J. L. Ba, "Adam: A method for stochastic gradient descent," in *Proc. ICLR: Int. Conf. Learn. Representations*, 2015, pp. 1–15.

[28] S. Wang, S. Bi, and Y.-J. A. Zhang, "Locational detection of the false data injection attack in a smart grid: A multilabel classification approach," *IEEE Internet Things J.*, vol. 7, no. 9, pp. 8218–8227, Sep. 2020.

Tong Wu (Member, IEEE) received the Ph.D. degree in information engineering from The Chinese University of Hong Kong, Hong Kong, in 2021. He is currently a Postdoctoral Associate with Cornell Tech, Cornell University, New York City, NY, USA. His research focuses on convex optimization in power systems.

Ying-Jun Angela Zhang (Fellow, IEEE) is currently a Professor with Department of Information Engineering, The Chinese University of Hong Kong, Hong Kong. Her research interests include optimization and learning in wireless communication systems and smart power grids. She is currently an Associate Editor-in-Chief of the IEEE Open Journals of the Communication Society and a Member of IEEE ComSoc Fellow Evaluation Committee. Previously, she was the Chair of the Executive Editorial Committee of the IEEE Transactions on Wireless Communications and the Founding Chair of IEEE ComSoc Smart Grid Communications Technical Committee. She was the co-recipient of the 2011 Marconi Prize Paper Award in Wireless Communications, the 2013 SmartGridComm Best Paper Award, and the 2014 IEEE ComSoc Asia Pacific Board Outstanding Paper Award. She won the 2006 Hong Kong Young Scientist Award as the only winner from engineering science. She is a Distinguished Lecturer of IEEE ComSoc.

Shuoyao Wang (Member, IEEE) received the B.Eng. (with first class Hons.) and the Ph.D. degrees in information engineering from The Chinese University of Hong Kong, Hong Kong, in 2013 and 2018, respectively. From 2018 to 2020, he was an Senior Researcher with Tencent, Shenzhen, China. Since 2020, he has been with the College of Electronic and Information Engineering, Shenzhen University, Shenzhen, where he is currently an Assistant Professor. His research interests include operational research and machine learning in multimedia processing, smart grid, and wireless communications.